

AI Agent Compromise — Incident Response Playbook

When the agent itself — its identity, memory, or supply chain — is owned

IR-01 covers the agent that was **tricked**: injected instructions in content it read. This playbook covers the agent that is **owned** — its credentials stolen, its identity hijacked, its persistent memory implanted, its tools or configuration trojaned. The distinction matters operationally: an injection incident is about untrusted *content*; a compromise incident is about untrusted *identity and state*. Treat it like any other identity compromise — with the twist that this identity holds live tool access and a memory that can re-infect a clean deployment. The SOC muscle memory for a compromised service account applies almost entirely; this runbook maps it onto agents, following the NIST 800-61 lifecycle.

The five compromise classes

Classify the incident first — each class has a different containment order and a different first signal. A single incident often spans two classes (e.g. a stolen token used to plant memory persistence).

Class	What happened	Tell-tale first signal
1 · Credential / token theft	The agent's API keys, OAuth tokens, or service credentials are stolen and used by the attacker	Agent identity authenticating from an origin, time, or pattern outside its baseline
2 · Identity hijack / rogue agent	Attacker acts as the agent — or registers a rogue agent — to inherit its trust and permissions	Agent activity with no matching user session or orchestrator record
3 · Memory / state persistence	A payload is implanted in persistent memory, threads, or vector state so it survives sessions	Cross-session behavior drift with no config change; memory writes outside a live session
4 · Tool / supply-chain compromise	A tool, plugin, or integration the agent calls is malicious, trojaned, or silently replaced	Tool registered or modified with no change record; tool responses steering behavior
5 · Orchestrator / config compromise	System prompt, agent configuration, or the orchestration layer itself is altered	Prompt/config version change outside the deployment pipeline

Classify and set severity

Severity is keyed to **identity blast radius**: everything the compromised identity can reach — tools, data, and other agents that trust it — for the full window of compromise, not just what the attacker has touched so far.

Severity	Trigger	Clock
SEV-1 Critical	Compromised identity can reach high-impact tools (send / delete / pay / execute), other agents trust it, or sensitive data is in scope	Immediate page
SEV-2 High	Compromise confirmed; access limited to low-impact tools or read-only data	< 1 hour
SEV-3 Medium	Strong indicators (anomalous auth, unexplained config change) but compromise unconfirmed	Same day
SEV-4 Low	Failed credential abuse attempts / probing of the agent's identity surface	Triage queue

PHASE 1 — Detect & analyze

Establish that this is identity/state compromise, not injection — provenance points to *who acted and what changed*, not to content the agent read. Then size the window.

- Confirm the class (1–5): pull agent auth logs, tool-call logs, memory/vector write logs, and config/prompt version history.
- Establish the window of compromise: first anomalous authentication or unauthorized change → containment. Assume everything in the window is attacker-influenced.
- Map identity blast radius: every tool, credential, data store, and downstream agent the identity can reach — not only what was demonstrably used.
- Correlate agent activity against orchestrator and user-session records — actions with no originating session are attacker actions.
- Check for persistence: memory/vector entries written in the window, scheduled or trigger-based tasks, modified tools, altered config.
- Determine whether the compromise is spreading: does any other agent or service trust this identity or consume its outputs?

PHASE 2 — Contain

Containment is revocation-first, in this order — kill the identity before you touch the state, or the attacker rebuilds persistence while you clean it.

- Trigger the kill switch: disable the agent endpoint and its orchestrator entry so no new sessions start.
- Revoke the agent's identity: disable the service principal / machine identity, revoke and rotate every token and key it holds. Do not merely rotate — revoke first, reissue later.
- Cut tool bindings at the gateway (send / delete / pay / execute), not in the prompt — a compromised agent ignores prompts.
- Freeze — do not yet purge — memory, threads, vector state, and config: they are evidence. Snapshot before any cleanup.
- Suspend trust relationships: block other agents and services from accepting this identity or consuming its queued outputs.
- If class 4 (tool / supply chain): disable the tool binding for **every** agent that uses it, not just this one.

PHASE 3 — Eradicate & recover

The identity is dead; now remove what the attacker left and rebuild. The rule: **nothing from the window of compromise survives into the rebuilt agent** — not the credential, not the memory, not the config.

- Root-cause the entry: how was the credential stolen / identity registered / tool replaced / config changed? Close that path first.
- Purge all memory, thread, and vector-state writes from the window; verify no residual instruction or implant remains.
- Rebuild identity from scratch: new unique identity, least privilege re-derived from current need — never restore the old credential or its old scope.
- Reinstate tools one at a time from verified sources, each behind its approval gate; verify integrity of any class-4 tool before rebinding.
- Restore config/prompt from a known-good, version-controlled source — not from the running (tainted) instance.
- Re-run the full assessment (intake → eight domains → risk tier) before restore; a No-Go stands until remediated.
- Return to production under heightened monitoring; alert on any recurrence of the window's indicators.

PHASE 4 — Post-incident

- Write the timeline: entry class, window of compromise, identity blast radius, actions fired, dwell time, containment gaps.
- Convert the compromise into detection rules (below) and red-team cases; test that the new detection fires on the original evidence.
- Re-score the agent's risk tier — a compromised agent's autonomy axis was, by definition, under-controlled.
- If class 4: run the supplier / tool review across the whole estate and update the tool approval inventory.
- Feed lessons to the owning team and update the crosswalk finding; schedule the re-assessment trigger.

Detection ideas for the SIEM (illustrative — adapt to your log schema)

These require the logging baseline from Playbook 04 (identity, tool calls, memory writes, config versions, session correlation). Tag each with its ATLAS technique so agent incidents ride the existing SOC workflow.

Detection concept	Signal to hunt	ATLAS
Anomalous agent auth	Agent identity authenticating from a new origin, at an unusual time, or at machine-atypical volume	AML.T0012 (Valid Accounts)
Token use outside session	Agent credential used with no corresponding live session or orchestrator record	AML.T0012
Orphan agent activity	Tool calls or outputs attributed to the agent with no originating user or scheduled task	AML.T0012 / Agent techniques
Unrecorded tool change	Tool registered, replaced, or re-scoped with no change-management record	AI Agent Tool Poisoning
Out-of-session memory write	Memory / vector-store write outside a live session, or bulk edits to stored context	Memory Manipulation
Config drift	Prompt / agent-config version change outside the deployment pipeline	Modify AI Agent Configuration
Post-compromise egress	High-impact tool call whose payload overlaps sensitive context, inside the suspect window	AML.T0086 (Exfiltration via AI Agent Tool Invocation)

Illustrative Sentinel-style hunt (pseudo-KQL — not production):

```
AgentAuthLogs
| where TimeGenerated > ago(7d)
| where IdentityType == "agent"
| join kind=leftanti (AgentSessions
  | project SessionId, AgentId) // auth with no live session
  on AgentId
| union (AgentConfigLogs
  | where ChangeSource != "pipeline") // config change outside CI/CD
| project TimeGenerated, AgentId, SourceIP, EventType, ChangeSource
```

Evidence to collect & host-side correlation

Preserve before cleanup: agent auth and token-usage logs for the full window, snapshots of memory / thread / vector state and the config/prompt version in effect, tool-call logs with parameters, the tool or artifact itself for class 4, and the orchestrator's session records. Correlate with the EDR and identity provider: every agent action should have a matching host or IdP event. An agent action with no endpoint or IdP telemetry — or endpoint activity under the agent's identity that the agent did not log — is itself a finding, and often the hijack signal.

Framework mapping

Agent-compromise incidents map to MITRE ATLAS **AML.T0012** (Valid Accounts) for credential and identity abuse, **AML.T0086** (Exfiltration via AI Agent Tool Invocation) where data left via a tool, and the agent techniques (Memory Manipulation, AI Agent Tool Poisoning, Modify AI Agent Configuration) for persistence and supply-chain classes; OWASP Agentic **ASI** (identity, memory, and tool misuse items) and OWASP **LLM06** where excessive agency enabled impact; NIST AI RMF (Manage); NIST 800-61 for the lifecycle; ISO/IEC 42001; SOC 2 CC6 (logical access); and EU AI Act Art.14 & Art.12 (oversight and logging). One incident, mapped once, satisfies many obligations.

Reviewer's quick checklist

- Compromise class (1–5) identified; window of compromise established.
- Identity blast radius mapped: tools, data, and trusting agents/services.
- Kill switch fired; identity revoked (not just rotated); tool bindings cut at the gateway.
- Memory / state / config snapshotted as evidence before purge.
- Entry path root-caused and closed; nothing from the window survives the rebuild.
- Identity rebuilt from scratch with least privilege; full re-assessment before restore.
- Detections written, tested against the original evidence, and ATLAS-tagged.

Safety. This runbook is exercised with safe sample data in an authorized lab or staging environment only. Never use real credentials, tokens, or customer data, and never run adversarial probes against production. All test results are labeled as authorized testing.